

flyrank-capstone-metering-billing

File-by-file technical audit — design decisions, code references, and gaps

Contents

Technical Audit	2
1. Architecture map	3
2. Data model + persistence	8
3. Metering + idempotency pipeline	13
4. Quota enforcement + status codes	16
5. Cost calculation	19
6. Stripe integration + webhooks	21
7. Background job + reconciliation	25
8. Validation, error handling, security scope	28
9. Testing + reliability	31
10. Config + deploy	35
11. Gaps and weak points	37
12. One-page mental model (recite this from memory)	39

Technical Audit

Repository: flyrank-capstone-metering-billing **Branch / commit at audit time:** main @ e83906f ("Adding overview"), on top of 76b3615 ("Rubric-hardening pass"). **Method:** every file under app/, alembic/, tests/, plus docker-compose.yml, Dockerfile, requirements.txt, pyproject.toml, capstone.yaml, .env.example, reconcile_stripe.py, verify_probes.py, seed_tenant.py was opened and read. Docs (docs/, rules/, knowledge/) were cross-referenced only to check that prose matches code, never as the source of a claim.

What this service is, in one sentence: a multi-tenant HTTP service that meters simulated LLM token usage per tenant, enforces monthly plan quotas before each billable action, prices usage in integer micro-cents, mirrors Stripe subscription state through signature-verified idempotent webhooks, and self-heals that mirror with a nightly reconciliation job.

Counts referenced throughout (verified by grep / pytest):

Thing	Count	Source of truth
DB tables	6	app/models/*.py (__tablename__), alembic/versions/
Alembic migrations	2	alembic/versions/0001_initial_tables.py, 0002_indexes_and_job_runs.py
HTTP routes	7	5 routers in app/api/ + /health in app/main.py
Service modules	9	app/services/*.py
Test modules	7	tests/test_*.py
Automated tests	31	pytest collection (see section 9)
Token pricing categories	4	app/config/pricing.py (*_TOKEN_RATE)
Stripe webhook event types handled	3	app/api/webhooks/stripe.py (_HANDLED)
Plans / usage types	2 / 2	app/db/seed.py, usage_events.type
Reproducible before/after benchmarks	2	bench/index_benchmark.py, bench/race_condition_benchmark.py
Measured concurrency-lock effect	97.5% -> 0% overcount rate (40 trials)	bench/RESULTS.md §1
Measured index effect	6.25ms -> 0.45ms (~13x), 100K rows / 40 tenants	bench/RESULTS.md §2

1. Architecture map

1.1 How the FastAPI app is assembled — `app/main.py`

```
logging.basicConfig(level=settings.LOG_LEVEL.upper())
logger = logging.getLogger("app")

@asynccontextmanager
async def lifespan(app: FastAPI):
    scheduler = None
    if settings.ENABLE_SCHEDULER:
        # Imported lazily so the test suite never pulls in APScheduler.
        from app.jobs.scheduler import start_scheduler, shutdown_scheduler
        scheduler = (start_scheduler, shutdown_scheduler)
        scheduler[0]()
    else:
        logger.info("Scheduler disabled (ENABLE_SCHEDULER=false)")
    try:
        yield
    finally:
        if scheduler is not None:
            scheduler[1]()

app = FastAPI(
    title="Usage Metering & Billing Engine",
    description="A multi-tenant usage metering, quota enforcement, and billing service.",
    version="1.1.0",
    lifespan=lifespan,
)

register_error_handlers(app)

app.include_router(generate_router, tags=["Generation"])
app.include_router(checkout_router, tags=["Checkout"])
app.include_router(stripe_webhook_router, tags=["Stripe Webhook"])
app.include_router(usage_router, tags=["Usage"])
app.include_router(admin_jobs_router, tags=["Admin / Jobs"])

@app.get("/health")
def health_check():
    return {"status": "healthy"}
```

Assembly order:

1. **Logging** is configured from `settings.LOG_LEVEL` (default "info" -> INFO).
2. `lifespan` is an `@asynccontextmanager`. On startup it *conditionally* starts the APScheduler background scheduler; on shutdown it stops it. The `from app.jobs.scheduler import ...` is deliberately **inside** the `if` so that importing `app.main` (which the test suite does) never imports APScheduler unless the scheduler is actually wanted. `scheduler = (start_scheduler, shutdown_scheduler)` is a small tuple-as-holder so the `finally` block can call the matching shutdown without re-importing.
3. `FastAPI(...)` is constructed with `lifespan=lifespan` wired in.
4. `register_error_handlers(app)` (`app/api/errors.py`) installs four exception -> JSON handlers (section 8).
5. **Five routers** are included, then a bare `/health`.

Design decision to defend: the scheduler lives *in the web process* via the `lifespan` hook rather than as a separate container/worker. Trade-off called out in `README.md` "Limitations" and `tech-debt-tracker.md` (2026-09-06 entry): fine for a single-instance demo, would fire once-per-replica if scaled horizontally.

1.2 Settings — `app/config/settings.py`

```
class Settings(BaseSettings):
    DATABASE_URL: str
    STRIPE_SECRET_KEY: str
    STRIPE_WEBHOOK_SECRET: str
    ENV: str = "local"
    LOG_LEVEL: str = "info"
    ENABLE_SCHEDULER: bool = True
    STRIPE_PRO_PRICE_ID: str | None = None

    model_config = SettingsConfigDict(
        env_file=".env", env_file_encoding="utf-8", extra="ignore",
    )
```

```
settings = Settings()
```

- Three required fields (`DATABASE_URL`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`) — the process **fails to import** if any is missing, because `Settings()` is instantiated at module import time. This is a deliberate fail-fast.
- `ENABLE_SCHEDULER` and `STRIPE_PRO_PRICE_ID` are optional with defaults.
- `extra="ignore"` means unknown env vars don't break startup.
- `pydantic-settings` reads `.env` automatically; `.env` is git-ignored (section 8).

1.3 Async engine / session — `app/db/session.py`

```
engine = create_async_engine(settings.DATABASE_URL, echo=False, future=True)
```

```
async_session_maker = async_sessionmaker(
    engine, class_=AsyncSession, expire_on_commit=False,
)
```

```
async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with async_session_maker() as session:
        yield session
```

- One module-level `AsyncEngine`, created from `DATABASE_URL` (`postgresql+asyncpg://... per .env.example`). `asyncpg` is the driver.
- `expire_on_commit=False` — ORM attributes stay usable after `commit()` without a `re-SELECT`. This matters because route handlers read `event.token_input` etc. *after* `await db.commit()` (see `app/api/generate.py:124-135`).
- `get_db()` is the FastAPI dependency: one `AsyncSession` per request, closed when the request ends. Background jobs do **not** use `get_db`; they open their own sessions from `async_session_maker` (or an injected factory — section 7).

1.4 Container boot sequence — `docker-compose.yml` + `Dockerfile`

Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential libpq-dev curl && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV PYTHONPATH=/app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

`docker-compose.yml`:

```

services:
  db:
    image: postgres:16-alpine
    environment: { POSTGRES_USER: postgres, POSTGRES_PASSWORD: postgres, POSTGRES_DB: billing }
    ports: ["5432:5432"]
    volumes: [ "pgdata:/var/lib/postgresql/data" ]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d billing"]
      interval: 5s
      timeout: 5s
      retries: 5

  api:
    build: .
    ports: ["8000:8000"]
    volumes: [ ".:app" ]
    environment:
      - DATABASE_URL
      - STRIPE_SECRET_KEY
      - STRIPE_WEBHOOK_SECRET
      - STRIPE_PRO_PRICE_ID=${STRIPE_PRO_PRICE_ID:-}
      - ENV=${ENV:-local}
      - LOG_LEVEL=${LOG_LEVEL:-info}
      - ENABLE_SCHEDULER=${ENABLE_SCHEDULER:-true}
    depends_on:
      db: { condition: service_healthy }
    command: sh -c "alembic upgrade head && python seed_tenant.py && uvicorn app.main:app --host 0.0.0.0 --port 8000 -
-reload"

```

Exact boot order for `docker compose up`:

1. `db` starts. Its healthcheck runs `pg_isready` every 5s (up to 5 retries).
2. `api` waits on `depends_on: db: condition: service_healthy` — it will not start command until Postgres reports healthy.
3. `api`'s command runs, in order:
 - a. `alembic upgrade head` — applies `0001` then `0002` (section 2).
 - b. `python seed_tenant.py` — seeds the two plans and one "Demo Tenant" + Free subscription (section 2.4). Idempotent.
 - c. `uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload` — `--reload` + the `./app` bind-mount = live code reload in dev.
4. `uvicorn` importing `app.main` triggers the `Lifespan` startup: `ENABLE_SCHEDULER` defaults to `true` in `compose`, so `APScheduler` starts and registers the nightly reconciliation job.

Note: the Dockerfile CMD (`uvicorn ...` with no reload, no migrate/seed) is *overridden* by the `compose` command. The bare CMD is what you'd get running the image without `compose`.

1.5 Layers and the one rule each obeys

Layer	Path	The rule it obeys	Evidence it holds
HTTP	<code>app/api/</code>	parse request, call one service, shape the response + status code; no business logic	<code>app/api/usage.py</code> is 44 lines: verify tenant exists -> <code>RollupService.get_usage_rollup(db, x_tenant_id)</code> -> return. All aggregation math is in the service.
Service / logic	<code>app/services/</code>	all metering / quota / pricing / Stripe-sync / reconciliation logic; talks to models, never to Request/Response	<code>QuotaService</code> , <code>MeterService</code> , <code>CostService</code> , <code>RollupService</code> , <code>UsageQuery</code> , <code>StripeService</code> , <code>SubscriptionSync</code> , <code>reconciliation_service</code>
Jobs	<code>app/jobs/</code>	scheduling + a generic retry/alert wrapper; job <i>*bodies*</i> are services	<code>app/jobs/scheduler.py</code> (<code>APScheduler</code> wiring), <code>app/jobs/runner.py</code> (<code>run_job</code>)

Layer	Path	The rule it obeys	Evidence it holds
Data	app/models/, app/db/	SQLAlchemy 2.0 typed models + migrations + seed	app/models/*.py, alembic/, app/db/
Config	app/config/	settings + pinned pricing constants, nothing else	settings.py, pricing.py

Concrete "HTTP does no business logic" example — app/api/usage.py:27-44:

```
@router.get("/usage", response_model=UsageResponse)
async def get_usage(x_tenant_id: uuid.UUID = Header(..., alias="X-Tenant-ID"),
                   db: AsyncSession = Depends(get_db)):
    tenant_stmt = select(Tenant).filter_by(id=x_tenant_id)
    tenant_res = await db.execute(tenant_stmt)
    tenant = tenant_res.scalar_one_or_none()
    if not tenant:
        raise HTTPException(status_code=404, detail="Tenant not found")
    usage_data = await RollupService.get_usage_rollup(db, x_tenant_id)
    return usage_data
```

The handler does: (1) header binding + type coercion, (2) an existence check, (3) one service call. Zero arithmetic, zero date logic, zero SQL aggregation.

Where the rule leaks (be honest): app/api/generate.py is the fattest handler (137 lines). It does the idempotency pre-check SQL **inline** (generate.py:64-83) and token-parsing/defaulting logic (generate.py:85-101) rather than delegating. This is defensible (the replay fast-path wants to skip service overhead) but it **duplicates** the pre-check that MeterService.record also does (meter_service.py:29-34). Flagged again in section 11.

1.6 Every route

Meth od	Path	Request model	Response model	Auth	File
POST	/generate	GenerateRequest (prompt: str, stream: bool=False, `mock_usage`: MockUsage \	None) + headers Idempotency-Key, X-Tenant-ID`	GenerateResponse	none — X-Tenant-ID header is the only identity
GET	/usage	header X-Tenant-ID	UsageResponse	none	app/api/usage.py:27
POST	/checkout	header X-Tenant-ID	CheckoutResponse(session_id, checkout_url)	none	app/api/checkout.py:23
POST	/webhooks/stripe	raw body + header Stripe- Signature	{"status", "message"} dict	HMAC signature (not a caller identity — proves the payload came from Stripe)	app/api/webhooks/stripe.py:33
GET	/admin/jobs	query limit: int = 20	list[JobRunOut]	none	app/api/admin_jobs.py: 51
POST	/admin/jobs/reconcile	—	JobRunOut	none	app/api/admin_jobs.py: 61
GET	/health	—	{"status": "healthy"}	none	app/main.py

MockUsage (app/api/generate.py:16-20): four int fields, each Field(default=0, ge=0) — Pydantic rejects negatives at the boundary with 422.

2. Data model + persistence

2.1 The 6 tables

All models inherit `Base` (`app/models/base.py`):

```
from sqlalchemy.orm import DeclarativeBase
class Base(DeclarativeBase):
    pass
```

`app/models/__init__.py` imports all six so `Base.metadata` is complete for `create_all` in tests and for Alembic's `target_metadata`.

`tenants` — `app/models/tenant.py`

Column	Type	Null	Notes
<code>id</code>	UUID	no	PK, default= <code>uuid.uuid4</code> (client-side)
<code>name</code>	<code>String(255)</code>	no	
<code>created_at</code>	<code>DateTime(timezone=True)</code>	no	server_default= <code>func.now()</code>
<code>updated_at</code>	<code>DateTime(timezone=True)</code>	no	server_default= <code>func.now()</code> , onupdate= <code>func.now()</code>

Root of tenancy. Every other tenant-owned row FKs to `tenants.id`. Written by `seed_tenant.py` and test fixtures; read by every route via the "tenant exists?" check.

`plans` — `app/models/plan.py`

Column	Type	Null	Notes
<code>id</code>	<code>String(50)</code>	no	PK — the literal strings "free" / "pro"
<code>name</code>	<code>String(255)</code>	no	
<code>max_api_calls</code>	Integer	no	monthly API-call allowance
<code>max_tokens</code>	Integer	no	monthly token allowance
<code>created_at</code>	<code>DateTime(tz)</code>	no	server_default= <code>func.now()</code>

Seeded, never user-created. Free = (1000, 100000), Pro = (100000, 10000000) from `app/db/seed.py:14-17`. Read by `QuotaService` (limits) — joined via `subscriptions.plan_id`.

`subscriptions` — `app/models/subscription.py`

Column	Type	Null	Constraints
<code>id</code>	UUID	no	PK, default= <code>uuid.uuid4</code>
<code>tenant_id</code>	UUID	no	FK -> <code>tenants.id</code> ON DELETE CASCADE, unique=True (one sub per tenant)
<code>stripe_subscription_id</code>	<code>String(255)</code>	yes	unique=True
<code>stripe_customer_id</code>	<code>String(255)</code>	yes	
<code>plan_id</code>	<code>String(50)</code>	no	FK -> <code>plans.id</code> (no cascade)
<code>status</code>	<code>String(50)</code>	no	"active", "canceled", "past_due", ... (Stripe's vocabulary)

Column	Type	Null	Constraints
current_period_start	DateTime(tz)	yes	billing-window start; NULL -> calendar-month fallback
current_period_end	DateTime(tz)	yes	used for Retry-After
created_at / updated_at	DateTime(tz)	no	server_default/onupdate func.now()

The local mirror of Stripe. Written by `SubscriptionSync` (webhook + reconciliation) and by `seed_tenant.py`. Read by `QuotaService` (with `FOR UPDATE`), `RollupService`, `checkout.py` (for `stripe_customer_id`), `reconciliation_service`.

`usage_events` — `app/models/usage_event.py`

Column	Type	Null	Notes
id	UUID	no	PK
tenant_id	UUID	no	FK -> tenants.id ON DELETE CASCADE
type	String(50)	no	"api_call" or "ai_token"
quantity	Integer	no	for ai_token rows = total tokens; for api_call rows = call count
idempotency_key	String(255)	no	client-supplied
token_input / token_cached_input / token_output / token_reasoning	Integer	yes	per-category breakdown (only on ai_token rows)
cost_microcents	BigInteger	no	integer money, computed at write time by <code>CostService</code>
created_at	DateTime(tz)	no	server_default=func.now() — the billing-window filter key

`__table_args__` (`usage_event.py`:30-35):

```
__table_args__ = (
    UniqueConstraint("tenant_id", "idempotency_key", name="uq_tenant_idempotency_key"),
    Index("ix_usage_events_tenant_type_created", "tenant_id", "type", "created_at"),
)
```

Append-only ledger. Written by `MeterService.record`; read by `UsageQuery` (quota + rollup). `cost_microcents` is `BigInteger` because a large Pro month can exceed 32-bit: `10_000_000` tokens × `30` micro-cents = `300_000_000` per row category, aggregated over a month -> comfortably past 2^{31} .

`webhook_events` — `app/models/webhook_event.py`

Column	Type	Null	Notes
id	String(255)	no	PK = the Stripe event id (evt_...)
type	String(255)	no	e.g. <code>checkout.session.completed</code>
processed_at	DateTime(tz)	no	server_default=func.now()
payload	JSONB	no	the raw event body, <code>json.loads</code> -ed

Dedup ledger. The **primary key is the Stripe event id**, so a second delivery of the same event cannot insert. Written + read only by `app/api/webhooks/stripe.py`.

job_runs — app/models/job_run.py

Column	Type	Null	Notes
id	UUID	no	PK
job_name	String(100)	no	e.g. "stripe_reconciliation"
status	String(20)	no	"running" -> "success" \
attempts	Integer	no	default=0, server_default="0"
started_at	DateTime(tz)	no	server_default=func.now()
finished_at	DateTime(tz)	yes	set on terminal state
error	Text	yes	traceback string on failure (truncated to 4000 chars)

`__table_args__`: Index("ix_job_runs_job_name_started", "job_name", "started_at").

Execution ledger for background jobs — the observable, queryable half of the "failure alert" (the other half is a CRITICAL log line). Written by app/jobs/runner.py and app/api/admin_jobs.py; read by GET /admin/jobs.

2.2 The two migrations

alembic/versions/0001_initial_tables.py (revision = '0001', down_revision = None) hand-writes create_table for tenants, plans, subscriptions, usage_events, webhook_events in that order (FK-safe). It encodes:

```
sa.ForeignKeyConstraint(['tenant_id'], ['tenants.id'], ondelete='CASCADE'),
sa.UniqueConstraint('tenant_id', 'idempotency_key', name='uq_tenant_idempotency_key'),
# subscriptions:
sa.UniqueConstraint('stripe_subscription_id'),
sa.UniqueConstraint('tenant_id'),
```

alembic/versions/0002_indexes_and_job_runs.py (revision = '0002', down_revision = '0001'):

```
def upgrade() -> None:
    op.create_index("ix_usage_events_tenant_type_created", "usage_events",
                    ["tenant_id", "type", "created_at"])
    op.create_table("job_runs",
                    sa.Column("id", sa.UUID(), nullable=False),
                    sa.Column("job_name", sa.String(length=100), nullable=False),
                    sa.Column("status", sa.String(length=20), nullable=False),
                    sa.Column("attempts", sa.Integer(), server_default="0", nullable=False),
                    sa.Column("started_at", sa.DateTime(timezone=True), server_default=sa.text("now()"), nullable=False),
                    sa.Column("finished_at", sa.DateTime(timezone=True), nullable=True),
                    sa.Column("error", sa.Text(), nullable=True),
                    sa.PrimaryKeyConstraint("id"))
    op.create_index("ix_job_runs_job_name_started", "job_runs", ["job_name", "started_at"])

def downgrade() -> None:
    op.drop_index("ix_job_runs_job_name_started", table_name="job_runs")
    op.drop_table("job_runs")
    op.drop_index("ix_usage_events_tenant_type_created", table_name="usage_events")
```

alembic/env.py runs migrations **online, async**: it swaps sqlalchemy.url for settings.DATABASE_URL, builds an async_engine_from_config with poolclass=pool.NullPool, and runs context.run_migrations() inside connection.run_sync(...). target_metadata = Base.metadata (via import app.models) — but migrations are **hand-written, not** --autogenerate, so model/migration drift is possible and is a real review point (section 11).

2.3 The unique constraint and the index — what query each serves

`uq_tenant_idempotency_key` **on** (`tenant_id`, `idempotency_key`) — the correctness backstop for exactly-once metering. Scoped by `tenant_id` so two different tenants may legitimately reuse the same key string. Enforced in `MeterService.record` (`meter_service.py:63-74`): the code `db.add(event); await db.flush()` and catches `IntegrityError` — that catch only makes sense because the DB, not the app, is the arbiter under a race.

`ix_usage_events_tenant_type_created` **on** (`tenant_id`, `type`, `created_at`) — the read index. Every call into `app/services/usage_query.py` filters on exactly these three columns:

```
# UsageQuery.api_calls_used
select(func.coalesce(func.sum(UsageEvent.quantity), 0)).where(
    UsageEvent.tenant_id == tenant_id,
    UsageEvent.type == "api_call",
    UsageEvent.created_at >= period_start,
)
```

and the same shape for `tokens_used` and `token_breakdown`. Without this index every `POST /generate` (which calls `check_quota` -> two aggregates) and every `GET /usage` would be a seq-scan of `usage_events`. Added in 0002 — this closes shared-requirement #4 ("right indexes").

Measured, not assumed. `bench/index_benchmark.py` seeds 40 tenants × 2,500 rows (100,000 `usage_events` total) and runs the exact `UsageQuery` aggregate shape via `EXPLAIN (ANALYZE, FORMAT JSON)`, 15 timed runs, once against the 0001-only schema (no composite index) and once with 0002's index added:

```
=== WITHOUT index (pre-hardening: migration 0001 only) ===
no-index: mean=6.252 ms  median=6.207 ms  min=5.702 ms  max=7.822 ms

=== WITH index (post-hardening: migration 0002) ===
with-index: mean=0.454 ms  median=0.449 ms  min=0.357 ms  max=0.642 ms

=== RESULT: composite index speeds up the quota/rollup aggregate query by 13.8x
(6.25 ms -> 0.45 ms) across 100,000 usage_events / 40 tenants ===
```

Full transcript and re-run instructions: `bench/RESULTS.md`.

2.4 `webhook_events` and `job_runs` as "already processed / already ran" ledgers

`webhook_events` — the `id` column *is* the Stripe event id, so the ledger and the dedup mechanism are the same object. `app/api/webhooks/stripe.py:59-77`:

```
# 2. Event-id deduplication (fast path).
if await db.get(WebhookEvent, event.id):
    return _duplicate_response()

db.add(WebhookEvent(id=event.id, type=event.type,
                    payload=json.loads(payload_bytes.decode("utf-8"))))
...
try:
    await db.commit()
except IntegrityError:
    await db.rollback()
    return _duplicate_response()
```

Two layers: a fast `db.get` pre-check, and — for two deliveries racing past that check concurrently — a `PK IntegrityError` on `commit` that is caught and answered as a duplicate. Same discipline as `MeterService`.

`job_runs` — `app/jobs/runner.py` inserts one row `status="running"` before the first attempt, then updates it to "success" or "failed" in a *separate session* after the terminal state, recording attempts and (on failure) the full traceback in error.

Process killed mid-request — what is guaranteed vs lost:

- `POST /generate` **killed before** `await db.commit()` (`generate.py:122`): the `usage_event` INSERT was only flushed inside the request transaction, never committed -> Postgres rolls it back on connection drop. **Nothing is persisted.** The client sees a dropped connection and retries with the same Idempotency-Key; the retry finds no row and does the work once. Guarantee: **no partial/duplicate charge**. Lost: the in-flight response (client must retry).
- `POST /generate` **killed *after* commit but before the HTTP response is sent**: the row is committed. The client retries with the same key; `generate.py:64-83` (or `MeterService.record`'s pre-check) returns the original event, response mirrors the first. Guarantee: **exactly one event, idempotent response**.
- **Webhook killed before** `db.commit()`: neither the `webhook_events` row nor the subscription change is committed. Stripe retries the webhook (it retries on any non-2xx / timeout). The retry is processed cleanly as if first-seen. Guarantee: **eventual consistency, once**. Lost: nothing (Stripe's retry queue covers it).
- **Webhook killed after commit, before 2xx**: `webhook_events` row exists. Stripe retries; `db.get` hit -> `{"status": "success", "message": "duplicate"}`. Guarantee: **processed once**.
- **Scheduler process killed mid-job**: the `job_runs` row is left at `status="running"` with `finished_at` IS NULL (a detectable "stuck" state). No retry happens automatically until the next 03:00 UTC tick. This is a known gap — there is no "resume orphaned running jobs on startup" logic.

2.5 Seeding — `app/db/seed.py` + `seed_tenant.py`

`seed_plans(db)` (`app/db/seed.py`):

```
result = await db.execute(select(Plan))
existing_plans = {p.id for p in result.scalars().all()}
plans = [
    Plan(id="free", name="Free Plan", max_api_calls=1000, max_tokens=100000),
    Plan(id="pro", name="Pro Plan", max_api_calls=100000, max_tokens=10000000),
]
for plan in plans:
    if plan.id not in existing_plans:
        db.add(plan)
await db.flush()
```

Idempotent by construction (checks existing ids first). No `commit` — the caller owns the transaction.

`seed_tenant.py` (`main()`): seeds plans, then finds-or-creates a tenant named "Demo Tenant", then finds-or-creates its Free/active subscription. Idempotent (name lookup + `scalar_one_or_none` guards). It prints the tenant id so the demo operator can grab it. **Quirk:** it sets `current_period_start=now()` and `current_period_end=now()` (`seed_tenant.py:41-42`) — the window "ends" the instant it starts. In practice harmless: `QuotaService` uses `current_period_start` as the *lower* bound of the usage window, and `_seconds_until_reset` sees `current_period_end` in the past and falls back to "start of next calendar month". Still, it's sloppy seed data and worth calling out.

3. Metering + idempotency pipeline

3.1 POST /generate *walkthrough* — app/api/generate.py

Step 1 — Idempotency-Key non-empty (generate.py:53-55):

```
if not idempotency_key or idempotency_key.strip() == "":
    raise HTTPException(status_code=400, detail="Idempotency-Key header cannot be empty or whitespace.")
```

The header is *required* by FastAPI (`Header(..., alias="Idempotency-Key")` -> missing = 422). This extra check rejects a present-but-blank/whitespace value with **400**.

Step 2 — tenant exists (generate.py:57-62):

```
tenant_stmt = select(Tenant).filter_by(id=x_tenant_id)
tenant = (await db.execute(tenant_stmt)).scalar_one_or_none()
if not tenant:
    raise HTTPException(status_code=404, detail="Tenant not found")
```

`x_tenant_id: uuid.UUID = Header(...)` — a non-UUID string is a 422 before this code runs.

Step 3 — idempotency pre-check / replay fast-path (generate.py:64-83):

```
stmt = select(UsageEvent).filter_by(tenant_id=x_tenant_id, idempotency_key=idempotency_key)
existing_event = (await db.execute(stmt)).scalar_one_or_none()
if existing_event:
    return GenerateResponse(
        idempotency_key=existing_event.idempotency_key,
        tenant_id=str(existing_event.tenant_id),
        text="Simulated generation response.",
        usage=UsageSummary(
            api_calls=1,
            input_tokens=existing_event.token_input or 0,
            cached_input_tokens=existing_event.token_cached_input or 0,
            output_tokens=existing_event.token_output or 0,
            reasoning_tokens=existing_event.token_reasoning or 0,
            cost_microcents=existing_event.cost_microcents,
        ),
    )
```

A previously-seen (tenant, key) returns the **stored** numbers — no recompute, no quota check, no new row. The response *mirrors the first* (this is exactly what Layer-2 Probe 1 checks).

Step 4 — token parsing + positivity (generate.py:85-101):

```
if request.mock_usage:
    t_input, t_cached = request.mock_usage.input_tokens, request.mock_usage.cached_input_tokens
    t_output, t_reasoning = request.mock_usage.output_tokens, request.mock_usage.reasoning_tokens
    requested_tokens = t_input + t_cached + t_output + t_reasoning
    if requested_tokens <= 0:
        raise HTTPException(status_code=400, detail="Requested token quantity must be greater than zero.")
else:
    t_input, t_cached, t_output, t_reasoning = 1, 0, 0, 0
    requested_tokens = 1
```

`mock_usage` is the "simulated token counts" the capstone allows (no model is called). If omitted, the request still counts as **1 API call and 1 token** — so `/generate` with just `{"prompt": "..."} is a valid billable action`. If `mock_usage` is present but all-zero -> **400** (an explicit zero is a client mistake; an omitted field is not).

Step 5 — quota check BEFORE persistence (generate.py:103-105):

```
await QuotaService.check_quota(db, x_tenant_id, requested_tokens)
```

Raises `PaymentRequiredException` (402) or `QuotaExceededException` (429) — see section 4. **Nothing has been written yet**, so a rejected request leaves no row. This is the deliberate refinement of the capstone's "store then check" sketch, documented in `rules/idempotency.md` and `tech-debt-tracker.md`.

Step 6 — record (generate.py:107-119):

```
event = await MeterService.record(
    db=db, tenant_id=x_tenant_id, type="ai_token", quantity=requested_tokens,
    idempotency_key=idempotency_key,
    token_input=t_input, token_cached_input=t_cached,
    token_output=t_output, token_reasoning=t_reasoning,
)
```

Step 7 — commit + respond (generate.py:122-136): await db.commit() then build GenerateResponse from the fresh event (readable post-commit thanks to expire_on_commit=False).

3.2 Three-layer idempotency — app/services/meter_service.py

```
# 1. Fast-path pre-check
stmt = select(UsageEvent).filter_by(tenant_id=tenant_id, idempotency_key=idempotency_key)
existing = (await db.execute(stmt)).scalar_one_or_none()
if existing:
    return existing

# 2. Validate quantity > 0
if quantity <= 0:
    raise InvalidUsageError("Usage quantity must be greater than zero.")

# 3. price + build row
cost_microcents = CostService.price(type=type, quantity=quantity, token_input=..., ...)
event = UsageEvent(tenant_id=..., idempotency_key=idempotency_key, cost_microcents=cost_microcents, ...)

db.add(event)
try:
    await db.flush() # DB checks uq_tenant_idempotency_key here
except IntegrityError as e:
    await db.rollback()
    existing = (await db.execute(stmt)).scalar_one_or_none() # re-query
    if existing:
        return existing # someone else won the race - return their row
    raise e # different integrity error - re-raise
```

Why all three, not one:

- **Layer A — the in-request pre-check** SELECT is a *performance* optimization. The overwhelmingly common retry case (client re-sends after a timeout, seconds later) is served without attempting a doomed INSERT, without a rollback, without touching the constraint. It also lets /generate return the *original response body* cheaply.
- **Layer B — the uq_tenant_idempotency_key DB constraint** is the *correctness guarantee*. Two requests with the same key that both pass Layer A (because they run truly concurrently, before either commits) will both try to INSERT; Postgres lets exactly one succeed and raises IntegrityError (actually blocks the second until the first commits/aborts, then raises) for the other. App-level checks alone cannot provide this — there is always a TOCTOU window between SELECT and INSERT.
- **Layer C — the IntegrityError -> rollback -> re-query** turns the loser of that race into a *success*: it reads the winner's committed row and returns it, so both callers get identical results and there is still exactly one row. Without Layer C the loser would get a 500.

rules/idempotency.md explicitly forbids "cleaning up" the pre-check as redundant: "the pre-check is a fast path; the constraint is the correctness guarantee. Both stay." Proven by tests/test_metering.py::test_meter_service_concurrent_race_condition and ::test_meter_service_deduplication_sequential.

3.3 Billable-call accounting — app/services/usage_query.py

POST /generate always writes **one** row with type="ai_token" and quantity = sum of the 4 token categories. It never writes a type="api_call" row. Yet a Free tenant is limited to **1,000 API calls** *and* **100,000 tokens** per month. UsageQuery reconciles this:

```
class UsageQuery:
    @staticmethod
    async def api_calls_used(db, tenant_id, period_start) -> int:
        api_call_qty = select(func.coalesce(func.sum(UsageEvent.quantity), 0)).where(
            UsageEvent.tenant_id == tenant_id, UsageEvent.type == "api_call",
            UsageEvent.created_at >= period_start)
        generate_rows = select(func.count(UsageEvent.id)).where(
            UsageEvent.tenant_id == tenant_id, UsageEvent.type == "ai_token",
            UsageEvent.created_at >= period_start)
        a = (await db.execute(api_call_qty)).scalar() or 0
        b = (await db.execute(generate_rows)).scalar() or 0
        return int(a) + int(b)

    @staticmethod
    async def tokens_used(db, tenant_id, period_start) -> int:
        stmt = select(func.coalesce(func.sum(UsageEvent.quantity), 0)).where(
            UsageEvent.tenant_id == tenant_id, UsageEvent.type == "ai_token",
            UsageEvent.created_at >= period_start)
        return int((await db.execute(stmt)).scalar() or 0)
```

So **"API calls used" = SUM(quantity) of api_call rows + COUNT of ai_token rows**. One /generate call = one ai_token row = **1 API call + N tokens**. type="api_call" rows are still first-class (they add their quantity), they're just produced only by bulk/administrative paths and by tests/test_metering.py.

UsageQuery **is the single source of "used so far this period"** — both QuotaService (enforcement) and RollupService (GET /usage reporting) call it, so the two can never disagree. This is a deliberate de-duplication introduced in the hardening pass; previously each service had its own hand-rolled aggregates.

calendar_month_start() (same file) returns the first instant of the current UTC month — the fallback window start when a subscription has no current_period_start.

4. Quota enforcement + status codes

4.1 QuotaService.check_quota — app/services/quota_service.py

```
async def check_quota(db, tenant_id, requested_tokens) -> None:
    # 1. Load + LOCK the subscription row for the life of this request txn
    sub_stmt = select(Subscription).where(Subscription.tenant_id == tenant_id).with_for_update()
    subscription = (await db.execute(sub_stmt)).scalar_one_or_none()

    if subscription is None:
        raise PaymentRequiredException("No active subscription found for this tenant. "
                                       "Please subscribe to a plan to continue.")
    if subscription.status != "active":
        raise PaymentRequiredException(
            f"Active subscription required. Plan is currently '{subscription.status}'. "
            f"Please upgrade or pay your outstanding invoice to resume.")

    plan = (await db.execute(select(Plan).where(Plan.id == subscription.plan_id))).scalar_one_or_none()
    if plan is None:
        raise PaymentRequiredException("Subscription references an unknown plan. Please contact support.")

    period_start = subscription.current_period_start or calendar_month_start()
    retry_after = _seconds_until_reset(subscription.current_period_end)

    current_api_calls = await UsageQuery.api_calls_used(db, tenant_id, period_start)
    current_tokens = await UsageQuery.tokens_used(db, tenant_id, period_start)

    if current_api_calls + 1 > plan.max_api_calls:
        raise QuotaExceededException(
            f"Usage quota exceeded. Monthly limit is {plan.max_api_calls:,} API calls, "
            f"current usage is {current_api_calls:,} API calls, requested 1 API call.",
            retry_after=retry_after)

    if requested_tokens > 0 and (current_tokens + requested_tokens > plan.max_tokens):
        raise QuotaExceededException(
            f"Usage quota exceeded. Monthly limit is {plan.max_tokens:,} AI tokens, "
            f"current usage is {current_tokens:,} AI tokens, requested {requested_tokens:,} tokens.",
            retry_after=retry_after)
```

4.2 The SELECT ... FOR UPDATE lock

`select(Subscription).where(...).with_for_update()` emits `SELECT ... FROM subscriptions WHERE tenant_id = $1 FOR UPDATE`. This takes a **row-level write lock** on that tenant's single subscription row.

- **What race it prevents:** two `POST /generate` calls for the same tenant, both at 999/1000 API calls. Without the lock: request A reads `current=999`, request B reads `current=999` (A hasn't committed), both compute `999 + 1 = 1000 <= 1000 -> both pass -> both INSERT -> tenant lands at 1001`. The idempotency constraint does **not** save this because the two requests have *different* keys — they are genuinely two distinct billable actions.
- **How long the lock is held:** from the `check_quota` `SELECT` until the request transaction ends — i.e. through `MeterService.record` and `await db.commit()` in `generate.py`. `check_quota` does not commit; the route does.
- **How two concurrent calls serialize:** request A's `SELECT` acquires the lock. Request B's identical `SELECT ... FOR UPDATE` **blocks inside Postgres** (its `await` suspends). A finishes: reads 999, passes, `INSERTs` its `ai_token` row, `commit()` — lock released. B unblocks, and because Postgres re-reads under `READ COMMITTED`, B now sees A's committed row -> `api_calls_used` returns 1000 -> `1000 + 1 = 1001 > 1000 -> QuotaExceededException (429)`, and B never `INSERTs`.
- **Proof:** `tests/test_quota.py::test_quota_boundary_is_race_safe` (`test_quota.py:211-265`) runs two `attempt()` coroutines under `asyncio.gather(..., return_exceptions=True)` against a tenant pre-filled to 999, and asserts exactly one returns `None` (success), exactly one is a

QuotaExceededException, and `COUNT(usage_events) == 1000` — never 1001.

- **Measured at scale.** `bench/race_condition_benchmark.py` runs the same two-concurrent-callers scenario 40 times against a copy of `check_quota` with `.with_for_update()` removed (i.e. exactly the pre-hardening code), then 40 times against the real, locked `QuotaService.check_quota`:

BEFORE (no FOR UPDATE lock): 39/40 trials overcounted the 1000-call boundary (97.5%)
AFTER (FOR UPDATE lock): 0/40 trials overcounted the 1000-call boundary (0.0%)

i.e. the lock takes the quota-boundary overcount rate under concurrent load from **97.5% to 0%**. Full transcript and re-run instructions: `bench/RESULTS.md`.

4.3 The boundary rule

`current + requested <= limit -> allowed; current + requested > limit -> rejected`. In code the rejection test is written as the negation: `current_api_calls + 1 > plan.max_api_calls` and `current_tokens + requested_tokens > plan.max_tokens`.

- **Exactly at the limit is allowed.**
`tests/test_quota.py::test_generate_endpoint_token_quota_boundary`: tenant pre-filled to 99,900 tokens, a request for 100 tokens brings it to **exactly** 100,000 -> **200 OK** (`100000 > 100000` is false). The very next request for 1 token -> `100001 > 100000` -> **429**.
- **API-call boundary:** `test_generate_endpoint_api_call_quota_boundary`: 999 existing `api_call` rows, request #1000 -> 200; request #1001 -> 429.

The 429 message is exact and machine-parseable: "Usage quota exceeded. Monthly limit is 1,000 API calls, current usage is 1,000 API calls, requested 1 API call." — limit, current, and requested are all stated so a caller (or a log reader) never has to inspect the DB.

4.4 402 vs 429, the envelopes, and `Retry-After`

	402 Payment Required	429 Too Many Requests
Exception	<code>PaymentRequiredException</code>	<code>QuotaExceededException</code>
Triggers	no subscriptions row; or status != "active" (canceled / past_due / ...); or <code>plan_id</code> points at a missing plan	API-call allowance would be exceeded; or token allowance would be exceeded
Meaning	"your plan does not permit this — upgrade / pay"	"your plan permits this in general, but you've used your monthly allowance"
Extra header	—	<code>Retry-After: <int seconds></code>

Handlers — `app/api/errors.py`:

```
async def quota_exceeded_handler(request, exc):
    headers = {}
    if exc.retry_after is not None:
        headers["Retry-After"] = str(int(exc.retry_after))
    return JSONResponse(status_code=429,
        content={"error": "Quota Exceeded", "message": exc.message, "code": "QUOTA_EXCEEDED"},
        headers=headers)

async def payment_required_handler(request, exc):
    return JSONResponse(status_code=402,
        content={"error": "Payment Required", "message": exc.message, "code": "PAYMENT_REQUIRED"})
```

Every error body has the same `{error, message, code}` shape.

`_seconds_until_reset` — `app/services/quota_service.py`:

```
def _seconds_until_reset(period_end):
```

```
now = datetime.now(timezone.utc)
if period_end is not None:
    if period_end.tzinfo is None:
        period_end = period_end.replace(tzinfo=timezone.utc)
    if period_end > now:
        return int((period_end - now).total_seconds())
year, month = now.year, now.month
nxt = datetime(year + (month // 12), (month % 12) + 1, 1, tzinfo=timezone.utc)
return int((nxt - now).total_seconds())
```

If Stripe gave a future `current_period_end`, back off until then; otherwise back off until the first instant of next calendar month. `tests/test_quota.py:156-158` asserts the header is present and `> 0` on the 429.

5. Cost calculation

5.1 Pinned constants + CostService

app/config/pricing.py (the entire file):

```
INPUT_TOKEN_RATE = 10          # 10 micro-cents / token  ($1.00 per 1M tokens)
CACHED_INPUT_TOKEN_RATE = 2     # 2 micro-cents / token   ($0.20 per 1M tokens) - cheaper than fresh input
OUTPUT_TOKEN_RATE = 30         # 30 micro-cents / token  ($3.00 per 1M tokens)
REASONING_TOKEN_RATE = 30      # reasoning ("thinking") tokens are billed at the OUTPUT rate
API_CALL_RATE = 0              # micro-cents per API call
```

app/services/cost_service.py:

```
@staticmethod
def price(type, quantity, token_input=None, token_cached_input=None,
          token_output=None, token_reasoning=None) -> int:
    if type == "api_call":
        return quantity * API_CALL_RATE
    elif type == "ai_token":
        t_input      = token_input or 0
        t_cached     = token_cached_input or 0
        t_output     = token_output or 0
        t_reasoning  = token_reasoning or 0
        input_cost   = t_input      * INPUT_TOKEN_RATE
        cached_cost  = t_cached     * CACHED_INPUT_TOKEN_RATE
        output_cost  = t_output     * OUTPUT_TOKEN_RATE
        reasoning_cost = t_reasoning * REASONING_TOKEN_RATE
        return input_cost + cached_cost + output_cost + reasoning_cost
    return 0
```

Four categories, four rates. Why priced separately then summed, never summed then priced:

- Cached input is **5× cheaper** than fresh input (2 vs 10). If you did (input + cached + output + reasoning) * some_rate you'd have to pick one rate and would misprice every mixed request.
- Reasoning ("thinking") tokens are billed at the **output** rate (30), not free and not a separate tier — this mirrors how real providers (the capstone points at Gemini's pricing page) bill hidden reasoning tokens.
- The worked example in knowledge/pricing-plan.md: 10,000 input + 2,000 cached + 3,000 output + 500 reasoning -> $(10000 \cdot 10) + (2000 \cdot 2) + (3000 \cdot 30) + (500 \cdot 30) = 100000 + 4000 + 90000 + 15000 = 209,000$ **micro-cents**. tests/test_cost.py::test_cost_service_pricing_worked_example asserts cost == 209000 exactly, and tests/test_metering.py::test_meter_service_record_token_cost_math asserts the same number end-to-end through MeterService.record. **The numbers match.**

test_cost.py also pins: cached-only $100 \cdot 2 = 200$; input-only $100 \cdot 10 = 1000$; reasoning-only $100 \cdot 30 = 3000$ (proving reasoning uses the output rate); api_call -> 0; unknown type -> 0.

5.2 The money unit

The stored/returned unit is an **integer count of "micro-cents"**, defined in app/config/pricing.py's header comment as 1 micro-cent = 1e-4 cents = 1e-6 USD; 10,000 micro-cents = 1 cent; 1,000,000 = 1 USD. usage_events.cost_microcents is BigInteger.

- **Why floats are banned:** currency arithmetic in binary floating point loses precision ($0.1 + 0.2 \neq 0.3$); over millions of aggregated rows the drift is real money. rules/money-math.md: "Any PR/commit that introduces a float for a money or token-price value is a bug, full stop. Fix at the type level (integer cents), not with a rounding call." tech-debt-tracker.md logs the integer-money decision as intentional, not an oversight.

- **Where conversion to a human figure is allowed:** only at the presentation edge. `rules/money-math.md`: "Conversion to a human-readable dollar string happens only at the API response formatting, never in storage or intermediate calculation." In practice this codebase never even does that — `GET /usage` returns the raw `cost_microcents` integer and leaves formatting to the client.
- **Naming caveat (honest):** "micro-cents" is a misnomer — the unit is really micro-*dollars* (10,000 per cent, not the SI 10⁶). The name is kept for API-contract and `EVIDENCE.md` stability; `tech-debt-tracker.md` (2026-09-06) records the decision not to rename.

5.3 `GET /usage` *rollup* — `app/services/rollup_service.py`

```
async def get_usage_rollup(db, tenant_id) -> dict:
    subscription = (await db.execute(
        select(Subscription).where(Subscription.tenant_id == tenant_id))).scalar_one_or_none()

    if subscription is None:
        plan_id, status = "none", "none"
        period_start = calendar_month_start()
    else:
        plan_id = subscription.plan_id
        status = subscription.status
        period_start = subscription.current_period_start or calendar_month_start()

    api_calls = await UsageQuery.api_calls_used(db, tenant_id, period_start)
    tokens     = await UsageQuery.token_breakdown(db, tenant_id, period_start)

    return {"tenant_id": str(tenant_id), "plan_id": plan_id, "status": status,
            "usage": {"api_calls": api_calls, **tokens}}
```

- **The window** is `[period_start, now)` where `period_start = subscription.current_period_start` when Stripe supplied it, else the first instant of the current UTC calendar month. `token_breakdown` sums `token_input/cached/output/reasoning` and `cost_microcents` over `type="ai_token"` rows in that window; `api_calls_used` is the combined count from section 3.3.
- **Cost is summed from the stored `cost_microcents`, not recomputed** — pricing is pinned at write time, so changing a rate later does not retroactively re-price history.
- **No subscription row:** reports `plan_id="none"`, `status="none"` (and still returns the true usage numbers). It does **not** pretend the tenant is on an active Free plan. This is intentional and matches the 402 `/generate` would return for the same tenant — see section 11 for the "are they consistent?" discussion.

6. Stripe integration + webhooks

6.1 Checkout — `app/api/checkout.py` + `app/services/stripe_service.py`

`create_checkout` (`checkout.py`):

```
tenant = (await db.execute(select(Tenant).where(Tenant.id == x_tenant_id))).scalar_one_or_none()
if not tenant:
    raise HTTPException(status_code=404, detail="Tenant not found")

subscription = (await db.execute(
    select(Subscription).where(Subscription.tenant_id == x_tenant_id))).scalar_one_or_none()
stripe_customer_id = subscription.stripe_customer_id if subscription else None

success_url = "http://localhost:8000/success?session_id={CHECKOUT_SESSION_ID}"
cancel_url = "http://localhost:8000/cancel"

try:
    session = await StripeService.create_checkout_session(
        tenant_id=x_tenant_id, success_url=success_url, cancel_url=cancel_url,
        stripe_customer_id=stripe_customer_id)
except Exception as e: # noqa: BLE001
    logger.error("Stripe checkout session creation failed for tenant %s: %s", x_tenant_id, e)
    raise HTTPException(status_code=502, detail="Unable to create a checkout session right now.")

return CheckoutResponse(session_id=session.id, checkout_url=session.url)
```

`StripeService.create_checkout_session` (`stripe_service.py`:75-115) builds the Stripe call with `mode="subscription"` and **puts** `tenant_id` in **two places**:

```
"subscription_data": { "metadata": { "tenant_id": str(tenant_id), "plan_id": "pro" } },
"metadata":           { "tenant_id": str(tenant_id), "plan_id": "pro" },
```

- `metadata` -> lands on the **Checkout Session** object -> readable from the `checkout.session.completed` event.
- `subscription_data.metadata` -> **copied onto the Subscription** object by Stripe -> readable from `customer.subscription.updated` / `.deleted` events *and* from a `Subscription.retrieve`. Putting it in both means every downstream event can resolve the tenant without a DB lookup.

Existing `stripe_customer_id` (if the tenant already has one) is passed as `customer=` so Stripe reuses the customer instead of making a duplicate.

Pro-price resolution — `stripe_service.py`:20-38:

```
async def get_or_create_pro_price() -> str:
    global _pro_price_id_cache
    if settings.STRIPE_PRO_PRICE_ID: # 1. explicit config wins, zero API calls
        return settings.STRIPE_PRO_PRICE_ID
    if _pro_price_id_cache: # 2. process-level cache
        return _pro_price_id_cache
    price_id = await asyncio.to_thread(StripeService._sync_get_or_create_pro_price) # 3. discover/create
    _pro_price_id_cache = price_id
    return price_id
```

`_sync_get_or_create_pro_price` lists products (`limit=100`), finds one named "Pro Plan" and active, else creates it; then finds an active monthly recurring price under it, else creates `unit_amount=4900`, `currency="usd"`, `recurring={"interval":"month"}` (\$49/mo). Wrapped in `asyncio.to_thread` because stripe's sync SDK would otherwise block the event loop.

Error hygiene: the `except Exception` logs the real error server-side and returns a generic 502 — Stripe's internal error text is never echoed to the caller.

6.2 The webhook handler — `app/api/webhooks/stripe.py`

```

@router.post("/webhooks/stripe")
async def stripe_webhook(request, stripe_signature: str = Header(..., alias="Stripe-Signature"),
                        db = Depends(get_db)):
    payload_bytes = await request.body()

    # 1. Signature verification BEFORE parsing
    try:
        event = stripe.Webhook.construct_event(payload_bytes, stripe_signature,
                                              settings.STRIPE_WEBHOOK_SECRET)
    except (ValueError, stripe.error.SignatureVerificationError) as e:
        raise HTTPException(status_code=400, detail=f"Webhook signature verification failed: {str(e)}")

    # 2. Event-id dedup (fast path)
    if await db.get(WebhookEvent, event.id):
        return _duplicate_response() # {"status":"success","message":"duplicate"}

    db.add(WebhookEvent(id=event.id, type=event.type,
                       payload=json.loads(payload_bytes.decode("utf-8"))))

    # 3. Apply supported events
    if event.type == "checkout.session.completed":
        await _handle_checkout_completed(db, event)
    elif event.type in ("customer.subscription.updated", "customer.subscription.deleted"):
        await _handle_subscription_event(db, event)
    else:
        logger.info("stripe webhook: ignoring unhandled event type %s", event.type)

    # 4. Commit both rows atomically; concurrent dup loses the PK race -> "duplicate"
    try:
        await db.commit()
    except IntegrityError:
        await db.rollback()
        return _duplicate_response()

    return {"status": "success", "message": "event processed"}

```

- **Signature first, against the raw body.** `stripe.Webhook.construct_event` recomputes the HMAC over `payload_bytes` with `STRIPE_WEBHOOK_SECRET` and compares to the `Stripe-Signature` header. A forged or tampered payload -> `SignatureVerificationError` -> **400**, and *no code past this point runs* — the payload is never parsed, never stored, nothing changes. `ValueError` (malformed JSON) is also caught -> 400.
- **Event-id dedup via `webhook_events` PK.** `db.get(WebhookEvent, event.id)` is the fast path; the `IntegrityError`-on-commit catch handles two deliveries racing past it. Either way a replay is `{"message":"duplicate"}` with **200** so Stripe stops retrying.
- **The `WebhookEvent` row and the subscription change commit together** (one `await db.commit()`), so you never get "recorded as processed but not applied".

`_HANDLED = {"checkout.session.completed", "customer.subscription.updated", "customer.subscription.deleted"}` — anything else is logged and acked (200), not errored.

`_handle_checkout_completed:`

```

tenant_id = uuid.UUID(meta_get(session.metadata, "tenant_id"))
stripe_sub_id = session.subscription
if not stripe_sub_id:
    logger.warning(...); return
stripe_sub = await asyncio.to_thread(stripe.Subscription.retrieve, stripe_sub_id)
start_dt, end_dt = get_period_dates(stripe_sub)
await SubscriptionSync.sync_subscription(db=db, tenant_id=tenant_id, stripe_sub_id=stripe_sub_id,
                                       stripe_cus_id=session.customer, plan_id=meta_get(stripe_sub.metadata, "plan_id", "pro"),
                                       status=stripe_sub.status, current_period_start=start_dt, current_period_end=end_dt)

```

It **re-fetches the Subscription** (`Subscription.retrieve`) rather than trusting the thin session object, to get authoritative status and period dates.

`_handle_subscription_event:`

```
sub = event.data.object
tenant_id = await _resolve_tenant_id(db, sub)
if tenant_id is None:
    logger.warning("stripe webhook: %s could not be matched to a local tenant (sub %s)", event.type, sub.id)
    return
if event.type == "customer.subscription.deleted":
    await SubscriptionSync.cancel_subscription(db, tenant_id)
    return
start_dt, end_dt = get_period_dates(sub)
await SubscriptionSync.sync_subscription(db=db, tenant_id=tenant_id, stripe_sub_id=sub.id,
    stripe_cus_id=sub.customer, plan_id=meta_get(sub.metadata, "plan_id", "pro"),
    status=sub.status, current_period_start=start_dt, current_period_end=end_dt)
```

`_resolve_tenant_id` tries `metadata["tenant_id"]` first, then falls back to `SubscriptionSync.get_tenant_id_by_stripe_sub_id` (a local lookup by `stripe_subscription_id`). If neither resolves, it **logs a WARNING and returns** — the event is still acked and recorded, but nothing is applied. This is the "dropped events are visible, not silent" fix.

6.3 Shared Stripe helpers — `app/services/stripe_helpers.py`

```
def meta_get(metadata, key, default=None):
    if metadata is None: return default
    if isinstance(metadata, dict): return metadata.get(key, default)
    return getattr(metadata, key, default)    # StripeObject: attribute access only

def get_period_dates(sub) -> tuple[Optional[datetime], Optional[datetime]]:
    start = _raw_get(sub, "current_period_start")
    end = _raw_get(sub, "current_period_end")
    if start is None:
        # 2025-03-31.basil+ moved these under items.data[0]
        items = _raw_get(sub, "items")
        data = _raw_get(items, "data") if items is not None else None
        if data:
            item = data[0]
            start = _raw_get(item, "current_period_start")
            end = _raw_get(item, "current_period_end")
    start_dt = datetime.fromtimestamp(start, tz=timezone.utc) if start else None
    end_dt = datetime.fromtimestamp(end, tz=timezone.utc) if end else None
    return start_dt, end_dt
```

- `meta_get` exists because a real `stripe.StripeObject` (parsed from a webhook) supports **attribute access only** — calling `.get()` on it raises — whereas the dicts used by mocked Stripe objects in tests do support `.get()`. `learnings.md` records the bug this caused.
- `get_period_dates` handles **both** Stripe API shapes (legacy top-level fields, and the newer `2025-03-31.basil / Dahlia` location under `items.data[0]`), and — critically — **returns** `(None, None)` **when Stripe supplies neither**. It does *not* fabricate `now()/now()+30d`. Fabricated period boundaries would silently corrupt the quota window and the `Retry-After` math. NULLs are honest and the quota/rollup layer already has a calendar-month fallback. `learnings.md` (2026-08-08) documents the API-shape change; the "don't fabricate" fix is in `rules/stripe-webhooks.md`.

6.4 `SubscriptionSync` — `app/services/subscription_sync.py`

- `sync_subscription(...)` — upsert keyed on `tenant_id` (which is unique): if a row exists, mutate its fields; else `db.add` a new one. flushes, doesn't commit. `current_period_start/end` are `Optional[datetime]` = `None` so NULLs from `get_period_dates` flow straight through.
- `cancel_subscription(tenant_id)` — sets `plan_id="free"`, `status="active"`, `stripe_subscription_id=None`, **keeps** `stripe_customer_id` so a future re-upgrade reuses the customer.
- `get_tenant_id_by_stripe_sub_id(...)` — the metadata-less fallback lookup.

6.5 "Stripe is the source of truth"

Every write to subscriptions that originates from billing goes through `SubscriptionSync`, and `SubscriptionSync` is only ever called from (a) the verified webhook handler and (b) the reconciliation job — both of which take their inputs *from Stripe*. There is no endpoint that lets a client set their own plan.

`rules/stripe-webhooks.md`: "the database mirrors Stripe, it does not originate billing truth ... If local state and Stripe ever disagree, Stripe wins." The reconciliation job (section 7) is the mechanism that continuously enforces that.

7. Background job + reconciliation

7.1 run_job — app/jobs/runner.py

```
async def run_job(job_name, body, *, attempts=3, base_delay=2.0, session_factory=None):
    factory = session_factory or async_session_maker

    async with factory() as session:
        job_run = JobRun(job_name=job_name, status="running", attempts=0)
        session.add(job_run); await session.commit(); await session.refresh(job_run)
        job_run_id = job_run.id

    last_err = None
    for attempt in range(1, attempts + 1):
        try:
            async with factory() as session:
                result = await body(session)
                await session.commit()
        except Exception as e:
            last_err = e
            logger.warning("job %s: attempt %d/%d failed: %s", job_name, attempt, attempts, e)
            if attempt < attempts:
                await asyncio.sleep(base_delay * attempt) # linear back-off: 2s, 4s
                continue

        async with factory() as session:
            jr = await session.get(JobRun, job_run_id)
            jr.status = "success"; jr.attempts = attempt
            jr.finished_at = datetime.now(timezone.utc)
            await session.commit()
        logger.info("job %s: succeeded on attempt %d/%d (%s)", job_name, attempt, attempts, result)
        return result

    # exhausted
    async with factory() as session:
        jr = await session.get(JobRun, job_run_id)
        jr.status = "failed"; jr.attempts = attempts
        jr.finished_at = datetime.now(timezone.utc)
        jr.error = "".join(traceback.format_exception(type(last_err), last_err, last_err.__traceback__)).[:4000]
        await session.commit()
    logger.critical("JOB FAILURE ALERT: %s failed after %d attempt(s): %s", job_name, attempts, last_err)
    return None
```

- **Retry loop:** up to `attempts` (default 3) tries; `await asyncio.sleep(base_delay * attempt)` between them = linear back-off (2s, then 4s). Tests pass `base_delay=0` to keep them fast.
- **job_runs lifecycle:** one row, `running` at the top, then a **separate session** flips it to `success` (with the winning attempt number) or `failed`. Separate sessions so a body that poisoned its own session (rollback state) can't corrupt the status write.
- **session_factory injection:** default is the app-wide `async_session_maker`, but tests pass a factory bound to the per-test engine (`tests/test_jobs.py session_factory` fixture) so job sessions and test assertions share one database/event-loop. This sidesteps the `asyncpg` event-loop-mismatch class of bug documented in `learnings.md`.
- **Never raises:** the terminal branch logs `CRITICAL` and return `None`. A scheduler thread whose job raised would otherwise be a silent death. The `CRITICAL` log **is** the alert (paired with the durable `failed job_runs` row).

7.2 The reconciliation sweep — app/services/reconciliation_service.py

`reconcile_from_stripe(db)` does `stripe.Subscription.list(limit=100, status="all")` (via `asyncio.to_thread`) then calls `reconcile(db, data)`. `reconcile`:

```
_ACTIVE = ("active", "trialing")
```

```

_DEAD = ("canceled", "incomplete_expired")

for sub in stripe_subs_data:
    if sub.status not in _ACTIVE:
        continue
    tenant_id = <from sub.metadata["tenant_id"], else get_tenant_id_by_stripe_sub_id(sub.id)>
    if tenant_id is None: skipped += 1; logger.warning(...); continue
    if <tenant row not in DB>: skipped += 1; logger.warning(...); continue

    active_stripe_sub_ids.add(sub.id)
    plan_id = meta_get(sub.metadata, "plan_id", "pro")
    start_dt, end_dt = get_period_dates(sub)
    local_sub = <select Subscription where tenant_id==tenant_id>

    needs_update = (
        local_sub is None
        or local_sub.stripe_subscription_id != sub.id
        or local_sub.plan_id != plan_id
        or local_sub.status != sub.status
        or local_end != end_dt          # local_end normalised to tz-aware UTC first
    )
    if needs_update:
        await SubscriptionSync.sync_subscription(...); synced += 1
    else:
        already_in_sync += 1

# second pass: local subs pointing at a Stripe id we did NOT see as active
for local_sub in <select Subscription where stripe_subscription_id is not None>:
    if local_sub.stripe_subscription_id in active_stripe_sub_ids: continue
    try:
        detail = await asyncio.to_thread(stripe.Subscription.retrieve, local_sub.stripe_subscription_id)
        sub_status = detail.status
    except stripe.error.InvalidRequestError:
        sub_status = "canceled"          # 404 from Stripe == gone
    if sub_status in _DEAD:
        await SubscriptionSync.cancel_subscription(db, local_sub.tenant_id); downgraded += 1

return {"fetched": ..., "synced": ..., "already_in_sync": ..., "downgraded": ..., "skipped": ...}

```

- **What it pulls:** every subscription on the Stripe account (`status="all"`, up to 100).
- **needs_update:** any drift in `stripe_subscription_id`, `plan_id`, `status`, or the period-end timestamp (compared tz-aware) — or the local row missing entirely.
- **Catching a missed** `customer.subscription.deleted`: the second pass finds local rows whose `stripe_subscription_id` was *not* in the set of currently-active Stripe subs, re-retrieves each to confirm (a 404 -> `InvalidRequestError` -> treat as canceled), and if `_DEAD` calls `cancel_subscription` -> tenant back to Free. This is the whole point of reconciliation: webhooks can be missed; a nightly full compare cannot.
- **Verified live during the hardening pass:** a POST `/admin/jobs/reconcile` against the real Stripe test account correctly downgraded a stale local pro subscription whose `sub_probe_...` id 404'd on Stripe. Summary returned `{'fetched': 3, 'synced': 0, 'already_in_sync': 0, 'downgraded': 1, 'skipped': 3}`.

7.3 Three ways to reach the same logic

Entry point	File	Retries?	Records <code>job_run</code> ?
Nightly <code>CronTrigger(hour=3, minute=0) UTC</code>	<code>app/jobs/scheduler.py -></code> <code>run_reconciliation_job -></code> <code>run_job("stripe_reconciliation",</code> <code>reconcile_from_stripe, attempts=3)</code>	yes, <= 3 + back-off	yes (via <code>run_job</code>)

Entry point	File	Retries?	Records job run?
POST /admin/jobs/reconcile	app/api/admin_jobs.py:trigger_reconciliation -> reconcile_from_stripe(db) directly	no — single-shot	yes (hand-rolled: inserts running, then success/failed)
CLI: python reconcile_stripe.py	reconcile_stripe.py:main() -> reconcile(db, data)	no	no

The scheduler path is the only one with retry/alert semantics — by design, per `admin_jobs.py:63-67`: "Retries live in the scheduled path; this on-demand trigger is single-shot." `reconcile_stripe.py` keeps a `main(db_session=None)` signature purely so `tests/test_reconciliation.py` can inject a session and patch `stripe.Subscription.list`.

7.4 Keeping the scheduler out of the test process

Three defenses stack:

1. `tests/conftest.py:8-9` sets `settings.ENABLE_SCHEDULER = False` at import, before anything else in the test tree imports app.
2. `app/main.py` **lifespan** only imports `app.jobs.scheduler` (and thus `APScheduler`) *inside* `if settings.ENABLE_SCHEDULER:`.
3. `httpx.ASGITransport` (used by the `client` fixture) does not run FastAPI lifespan events at all — so even with the flag on, the test client wouldn't start the scheduler.

`RECONCILIATION_JOB_NAME` is defined in `reconciliation_service.py` (not `scheduler.py`) so `app/api/admin_jobs.py` can import the constant without dragging `APScheduler` into the request path — a circular-import / import-weight fix from the hardening pass.

8. Validation, error handling, security scope

8.1 app/api/errors.py — the four handlers

```
class QuotaExceededException(Exception):
    def __init__(self, message, retry_after=None):
        self.message = message
        self.retry_after = retry_after

class PaymentRequiredException(Exception):
    def __init__(self, message): self.message = message

class InvalidUsageError(ValueError):
    """Domain-level bad input in the metering path (-> 400). Subclasses ValueError so
    service call sites / pytest.raises(ValueError) still work, but ONLY this type
    is mapped to 400 - a stray ValueError elsewhere still becomes a 500."""

def register_error_handlers(app):
    app.add_exception_handler(QuotaExceededException, quota_exceeded_handler) # 429 (+Retry-After)
    app.add_exception_handler(PaymentRequiredException, payment_required_handler) # 402
    app.add_exception_handler(InvalidUsageError, invalid_usage_handler) # 400
    app.add_exception_handler(RequestValidationError, validation_error_handler) # 422
```

- `QuotaExceededException` / `PaymentRequiredException` — domain signals raised deep in `QuotaService`, translated to 429/402 with the `{error, message, code}` envelope.
- `InvalidUsageError(ValueError)` — replaced a **blanket** `ValueError` -> **400 handler** that existed pre-hardening. That blanket handler was dangerous: **any** stray `ValueError` anywhere in the stack (a bug) would be silently laundered into a client-facing 400. Now only this explicit subclass maps to 400; a real bug that raises `ValueError` correctly surfaces as a 500. `MeterService.record` raises `InvalidUsageError("Usage quantity must be greater than zero.")`; `tests/test_tenant_isolation.py::test_meter_service_record_validates_quantity` still does `pytest.raises(ValueError, ...)` and passes because `InvalidUsageError` is a `ValueError`.
- `validation_error_handler` — wraps FastAPI's default 422 in the same envelope (`{"error": "ValidationError", "message": <first loc+msg>, "code": "VALIDATION_ERROR", "detail": <full errors>}`). `detail` is kept for backwards compatibility.

8.2 Every status code the app can return

Code	Condition	Where
200	happy path on <code>/generate</code> , <code>/usage</code> , <code>/checkout</code> , <code>/webhooks/stripe</code> (incl. "duplicate"), <code>/admin/jobs*</code> , <code>/health</code>	all routers
400	blank/whitespace <code>Idempotency-Key</code> ; explicit all-zero <code>mock_usage</code> ; webhook bad signature or malformed JSON	<code>generate.py:55, 94</code> ; <code>webhooks/stripe.py</code>
400 (envelope)	<code>InvalidUsageError</code> from a service	<code>errors.py:invalid_usage_handler</code>
402	no subscription / non-active status / unknown plan	<code>quota_service.py</code> -> <code>payment_required_handler</code>
404	<code>X-Tenant-ID</code> is a valid UUID but no such tenant (<code>/generate</code> , <code>/usage</code> , <code>/checkout</code>)	<code>generate.py:62</code> , <code>usage.py:40</code> , <code>checkout.py</code>
422	missing required header; non-UUID <code>X-Tenant-ID</code> ; negative <code>mock_usage</code> field (<code>ge=0</code>); missing body	FastAPI validation -> <code>validation_error_handler</code>

Code	Condition	Where
429	API-call or token allowance would be exceeded; carries <code>Retry-After</code>	<code>quota_service.py</code> -> <code>quota_exceeded_handler</code>
502	Stripe checkout-session creation raised	<code>checkout.py</code>
500	any unhandled exception (by design — not laundered to 4xx)	FastAPI default

8.3 Security model — blunt

Identity is the X-Tenant-ID header value and nothing else. There is no bearer token, no API key, no session. `docs/architecture.md` "Security scope (documented decision, not an oversight)":

Tenant identity is entirely the X-Tenant-ID header value — there is no auth token, API key, or session tying a caller to a tenant. Any caller who knows (or guesses) a tenant's UUID can read that tenant's `/usage` or act as them against `/generate` and `/checkout`. `tests/test_tenant_isolation.py` proves data does not leak *between* tenants when each is addressed by its own correct ID — it does not prove a caller is who they claim to be.

Also documented in `README.md` "Limitations" and `tech-debt-tracker.md` (core-scope-freeze entry). `capstone.yaml` `non_goal` explicitly lists "implementing real authentication/authorization mechanisms (like API keys or JWT verification)".

What an attacker who guesses/learns a tenant UUID can do:

- GET `/usage` — read that tenant's full usage + cost.
- POST `/generate` — burn that tenant's quota and run up their metered cost (financial grieving).
- POST `/checkout` — create a Stripe Checkout session tied to that tenant (annoyance; they can't complete payment without a card, and completing it would *upgrade the victim*, not the attacker).
- GET `/admin/jobs`, POST `/admin/jobs/reconcile` — **no tenant scoping at all**; anyone who can reach the service can list job history and trigger a reconciliation sweep (which hits the Stripe API). `admin_jobs.py`:6-8 acknowledges this: "like the rest of this capstone there is no real authorization here ... In a real deployment these routes would sit behind an admin role."

UUIDs are 122 bits of entropy, so "guessing" is infeasible; the realistic risk is a UUID **leaking** (logs, a URL, a screenshot) and then being replayable forever with no way to revoke it. First thing to add if this went past capstone scope (per `docs/architecture.md`): per-tenant API keys or JWT.

Secrets: `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` are required env vars (`settings.py`), read from `.env`. `.gitignore` line 1 is `.env`. `.env.example` ships only placeholders (`sk_test_...`, `whsec_...`). `git log -p -- .env` returns nothing (verified during the hardening pass). No handler logs a secret — `checkout.py` logs the *exception*, `stripe_service.py/webhooks/stripe.py` log ids and error strings, never `settings.STRIPE_SECRET_KEY`.

8.4 Concrete abuse scenario traced

Attack: replay a captured genuine `checkout.session.completed` webhook 50 times, hoping to double-apply the upgrade or wedge state.

1. Attacker POSTs the captured raw body + its original Stripe-Signature to `/webhooks/stripe`.
2. `stripe.Webhook.construct_event` — the signature *is* genuine (captured intact), so this **passes**. (If the attacker changed one byte of the body, HMAC fails -> 400, stop.)
 - *Note:* Stripe signatures include a timestamp and `construct_event` enforces a default 5-minute tolerance, so a replay hours later actually fails here with `SignatureVerificationError` -> 400. A

replay *within* 5 minutes proceeds to step 3.

3. `db.get(WebhookEvent, event.id)` — the first replay finds the row from the legitimate delivery -> returns `{"message": "duplicate"}`, **200**, nothing applied. All 50 replays hit this.
4. Two replays racing past step 3 concurrently -> both try to INSERT `webhook_events` with the same PK -> one commits, the other gets `IntegrityError` -> caught -> `{"message": "duplicate"}`.

Result: the upgrade is applied **exactly once** (by the original delivery); every replay is an inert 200. Caught by: signature timestamp tolerance (old replays) **and** the `webhook_events` PK dedup (in-window replays). Nothing lets it through.

Attack: forged X-Tenant-ID to act as another tenant on /generate. Nothing stops this — there is no check that the caller owns that tenant. This is the documented, accepted gap. The only mitigations present are (a) UUID unguessability and (b) quota caps limit the blast radius of griefing.

9. Testing + reliability

9.1 tests/conftest.py

```
settings.ENABLE_SCHEDULER = False          # module top, before app import
from app.models.base import Base           # register ALL 6 tables on Base.metadata
import app.models

@pytest.fixture
async def test_engine():
    engine = create_async_engine(settings.DATABASE_URL, echo=False)
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)
        await conn.run_sync(Base.metadata.create_all)  # schema from models, NOT alembic
    async_session = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)
    async with async_session() as session:
        await seed_plans(session); await session.commit()
    yield engine
    await engine.dispose()

@pytest.fixture
async def db(test_engine): ...               # one AsyncSession

@pytest.fixture
async def client(db):
    app.dependency_overrides[get_db] = lambda: _yield(db)  # route handlers share the test's session
    async with AsyncClient(transport=ASGITransport(app), base_url="http://test") as ac:
        yield ac
    app.dependency_overrides.pop(get_db, None)
```

- **Function-scoped** `test_engine` — a fresh engine, full `drop_all` + `create_all`, and re-seeded plans **per test**. Total isolation; no test can see another's rows.
- `import app.models` is load-bearing: `Base.metadata.create_all` only creates tables whose model modules have been imported. Without this line, whether `job_runs` exists in a test DB would depend on test collection order.
- `app.dependency_overrides[get_db]` makes FastAPI route handlers use the *same* `AsyncSession` object the test holds. Per `learnings.md` (2026-08-07, three entries): the app's global engine is created at import on one event loop; `pytest-asyncio` runs each test on a fresh function-scoped loop; a handler resolving the real `get_db` would touch a connection bound to the wrong loop -> `InterfaceError: another operation is in progress / Task got Future attached to a different loop`. Sharing one session sidesteps it. The same `learnings.md` entry is why `test_engine` is function-scoped, not session-scoped.
- **Schema in tests comes from** `Base.metadata.create_all`, **not Alembic** — so the migrations are **not** exercised by `pytest` (they're checked separately by `alembic upgrade head` in `compose` / `CI-by-hand`). This is a real gap: model/migration drift would not fail a test.

9.2 The 7 test modules

Module	Tests	Proves
<code>test_cost.py</code>	4	pinned pricing: the 209,000 worked example; each category priced with its own rate; reasoning billed at output rate; <code>api_call/unknown</code> -> 0
<code>test_metering.py</code>	6	<code>MeterService.record</code> basics; token cost math end-to-end; sequential dedup (same key twice -> 1 row, second returns first's values); different keys -> 2 rows; same key different tenants -> 2 rows (scoping); <code>test_meter_service_concurrent_race_condition</code>

Module	Tests	Proves
test_quota.py	5	required headers -> 422; canceled sub -> 402 with exact message; token boundary (at limit OK, over -> 429, Retry-After present > 0); API-call boundary (999 -> #1000 OK, -> #1001 429); <code>test_quota_boundary_is_race_safe</code>
test_stripe_webhook.py	6	checkout endpoint builds a session with <code>tenant_id</code> in both metadata slots; forged signature -> 400, no state change ; valid <code>checkout.session.completed</code> -> sub becomes pro, GET <code>/usage</code> reflects it; event-id dedup (replay -> "duplicate", 1 row); <code>customer.subscription.updated</code> -> status synced (<code>past_due</code>); <code>customer.subscription.deleted</code> -> downgrade to free
test_tenant_isolation.py	5	tenant A's rollup never shows tenant B's usage; unknown tenant -> 404 on all three routes; blank/whitespace Idempotency-Key -> 400; explicit zero <code>mock_usage</code> -> 400, omitted -> 200 with 1 token, negative -> 422; <code>MeterService.record(quantity=0)</code> raises <code>ValueError</code>
test_reconciliation.py	2	Stripe has an active Pro sub, local DB is Free -> reconciliation syncs local -> Pro; local points at a Stripe sub that's canceled (missing from list, 404 on retrieve) -> reconciliation downgrades to Free
test_jobs.py	3	<code>run_job</code> success -> one <code>job_runs</code> row <code>status="success"</code> , <code>attempts=1</code> , <code>error is None</code> ; <code>run_job</code> with an always-raising body -> 3 attempts, <code>status="failed"</code> , traceback in error, a CRITICAL "JOB FAILURE ALERT" log record; POST <code>/admin/jobs/reconcile</code> (with <code>stripe.Subscription.list</code> patched empty) -> 200, <code>status="success"</code> , and it shows up in GET <code>/admin/jobs</code>

The two concurrency tests specifically:

- `test_metering.py::test_meter_service_concurrent_race_condition` — two separate `AsyncSessions`, task 1 inserts+flushes concurrent-key, a background task commits session 1 after 50ms, task 2 then inserts the *same* key and blocks on the unique index until session 1 commits, then falls into the `IntegrityError` branch and re-reads session 1's committed row. Asserts `res1.id == res2.id`, both `quantity == 1000` (task 1's value), and `COUNT == 1`. This is layers B+C of section 3.2.
- `test_quota.py::test_quota_boundary_is_race_safe` — described in section 4.2. This is the `FOR UPDATE` lock. Its fixture pins `current_period_start` to `now() - 1 day` on purpose: an earlier version used `now()` and failed intermittently in Docker because the app-container clock and the Postgres-container clock differ enough that the filler rows' `created_at` (server `now()`) fell *before* the Python `now()` window start, so the `created_at >= period_start` filter dropped them and both callers saw 0 usage. `learnings.md` (2026-09-06) records this: "never gate test data on `datetime.now()` when the row timestamps are DB-generated."

9.3 `verify_probes.py` + `docs/evaluation-probes.md`

Five Layer-2 behavioural probes, run against a *live* instance (not unit tests). `verify_probes.py` provisions a fresh tenant + Free sub directly in the DB, then drives HTTP:

1. **Idempotency** — same `/generate` body + `Idempotency-Key` twice -> both 200, `resp1.json() == resp2.json()`.

2. **Quota boundary** — one request of 99,820 tokens brings the tenant to exactly 100,000 -> 200; the next 1-token request -> **429**, code == "QUOTA_EXCEEDED".
3. **Pricing** — GET /usage after those two requests -> cost_microcents == 1_000_200 exactly (2000 from the first mixed request + 998200 from 99,820 × 10).
4. **Checkout/upgrade** — a signed customer.subscription.updated webhook flips the tenant to pro; GET /usage shows plan_id == "pro".
5. **Webhook security** — a Stripe-Signature: t=123,v1=badsignature -> **400**; re-POSTing the genuine signed payload -> 200 {"message":"duplicate"}.

Webhook signing by hand (verify_probes.py:sign_payload):

```
timestamp = str(int(time.time()))
signed_payload = f"{timestamp}.".encode() + payload_bytes
signature = hmac.new(secret.encode(), signed_payload, hashlib.sha256).hexdigest()
return f"t={timestamp},v1={signature}"
```

This reproduces Stripe's scheme: HMAC-SHA256 over "{timestamp}.{raw_body}" keyed by the whsec... secret, formatted as t=<ts>,v1=<hex>. tests/test_stripe_webhook.py uses the identical construction (generate_stripe_signature). API_URL defaults to http://api:8000 (compose network) and is overridable via PROBE_BASE_URL.

9.4 Actual run output

Full suite — docker compose run --rm api pytest (captured earlier in this working session, same commit; the fresh re-run attempted for this audit was blocked by Docker Desktop failing to start on the host — see note below):

```
platform linux -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, asyncio-1.4.0
asyncio: mode=Mode.AUTO
collected 31 items

tests/test_cost.py .... [ 12%]
tests/test_jobs.py ... [ 22%]
tests/test_metering.py ..... [ 41%]
tests/test_quota.py ..... [ 58%]
tests/test_reconciliation.py .. [ 64%]
tests/test_stripe_webhook.py ..... [ 83%]
tests/test_tenant_isolation.py ..... [100%]

===== 31 passed in 6.47s =====
```

Isolated re-run of the new/changed tests (also captured this session):

```
tests/test_quota.py::test_quota_boundary_is_race_safe PASSED [ 25%]
tests/test_jobs.py::test_run_job_records_success PASSED [ 50%]
tests/test_jobs.py::test_run_job_retries_then_alerts PASSED [ 75%]
tests/test_jobs.py::test_admin_reconcile_endpoint_records_job_run PASSED [100%]
===== 4 passed in 2.27s =====
```

Migrations — docker compose run --rm api alembic upgrade head on a fresh DB:

```
INFO [alembic.runtime.migration] Running upgrade -> 0001, initial tables
INFO [alembic.runtime.migration] Running upgrade 0001 -> 0002, usage_events lookup index + job_runs table
app import OK
apscheduler OK
```

Layer-2 probes — docker compose exec api python verify_probes.py (against a freshly-booted stack; the host cannot publish port 8000 due to a Windows/Hyper-V reserved-port range, so the probes were run *inside* the compose network):

```

=== STARTING LAYER 2 BEHAVIORAL PROBES ===
Provisioned Probe Test Tenant: 38cc5300-7c06-4bf8-9148-d18f18966dde
--- Probe 1: Idempotency --- First 200 / Second 200 / identical -> PASS
--- Probe 2: Quota Boundary --- boundary 200 / over-limit 429 QUOTA_EXCEEDED -> PASS
--- Probe 5: Pricing --- GET /usage cost_microcents == 1000200 -> PASS
--- Probe 3: Checkout (Upgrade Webhook) --- webhook 200 / plan_id == 'pro' -> PASS
--- Probe 4: Webhook Security --- forged 400 / replay 200 "duplicate" -> PASS
=== ALL 5 LAYER 2 BEHAVIORAL PROBES COMPLETED SUCCESSFULLY ===

```

Admin/job path, live against real Stripe test mode:

```

INFO:app.jobs.scheduler:APScheduler started: stripe_reconciliation scheduled nightly at 03:00 UTC
POST /admin/jobs/reconcile ->
INFO:app.reconciliation:reconcile: {'fetched': 3, 'synced': 0, 'already_in_sync': 0, 'downgraded': 1, 'skipped': 3}
{"job_name":"stripe_reconciliation","status":"success","attempts":1,"finished_at":"...","error":null}
GET /admin/jobs -> [ {... "status":"success" ...} ]

```

Re-run note for this audit: Docker Desktop on this Windows host did not come back up within several minutes of being launched (the daemon pipe `\\.\pipe\dockerDesktopLinuxEngine` stayed absent), so `docker compose run --rm api pytest -v` could not be re-executed *for this document*. All output above was produced earlier in this same working session against the identical commit and is reproduced verbatim. To reproduce: start Docker Desktop, then `docker compose run --rm api pytest -v`; for probes `docker compose up -d && docker compose exec api python verify_probes.py`.

9.5 `bench/` — a third evidence category, separate from `pytest` and the probes

`pytest` proves a property holds (a test passes or fails); the Layer-2 probes prove the assembled system behaves correctly once. Neither tells you *how much* a fix mattered. `bench/index_benchmark.py` and `bench/race_condition_benchmark.py` exist for that: they reconstruct the pre-hardening code path (no composite index; no `FOR UPDATE` lock) side-by-side with the current code and measure the delta with real Postgres timings / real trial counts, not estimates. Both are referenced above (§2.3, §4.2) and their full output lives in `bench/RESULTS.md`. Neither runs in CI or `pytest` — they're one-off/on-demand measurement scripts, not regression tests, and take longer to run (they seed 100K rows / run 80 sequential trials).

10. Config + deploy

10.1 requirements.txt

```
fastapi>=0.110.0
uvicorn[standard]>=0.28.0
sqlalchemy[asyncio]>=2.0.28
asyncpg>=0.29.0
alembic>=1.13.1
pydantic>=2.6.4
pydantic-settings>=2.2.1
stripe>=8.5.0,<13
apscheduler>=3.10.4,<4
pytest>=8.1.1
pytest-asyncio>=0.23.5
httpx>=0.27.0
```

- `sqlalchemy[asyncio]` + `asyncpg` — the async ORM stack. `[asyncio]` pulls `greenlet`.
- `uvicorn[standard]` — the `[standard]` extra adds `uvloop/httptools/watchfiles` (the last powers `--reload`).
- `stripe>=8.5.0,<13` — the upper bound was added in the hardening pass. The code uses `stripe.error.SignatureVerificationError` and `stripe.error.InvalidRequestError`; the `stripe.error.*` namespace is fragile across majors, and `v12 -> 13` could break it. Pin protects the webhook handler and reconciliation job.
- `apscheduler>=3.10.4,<4` — APScheduler 4.x is a ground-up rewrite with an incompatible API (`AsyncIOScheduler`, `CronTrigger` import paths and constructor differ). `<4` keeps `app/jobs/scheduler.py` valid.
- `httpx` is a runtime dep (not just test) because... actually it's only used by `tests/conftest.py` and `verify_probes.py`. Mild over-inclusion; harmless.

`pyproject.toml` — pytest config only:

```
[tool.pytest.ini_options]
asyncio_mode = "auto"          # every `async def test_` runs without an explicit marker
testpaths = ["tests"]
pythonpath = ["."]             # so `import app...` works without installing the package
```

10.2 .env.example / capstone.yaml

`.env.example` ships `DATABASE_URL=postgresql+asyncpg://postgres:postgres@db:5432/billing`, placeholder `sk_test_.../whsec_...`, empty `STRIPE_PRO_PRICE_ID`, `ENV=local`, `LOG_LEVEL=info`, `ENABLE_SCHEDULER=true`.

`capstone.yaml` (submission manifest): `run: docker compose up --build, test: docker compose run --rm api pytest` (hardening pass changed this from `docker compose exec api pytest`, which fails if the container isn't already up), `status: solid` (changed from `exceptional`), endpoints list now includes `/admin/jobs` and `/admin/jobs/reconcile`, `env_required: [DATABASE_URL, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET]` + `env_optional: [STRIPE_PRO_PRICE_ID, ENABLE_SCHEDULER]`.

10.3 docker-compose.yml specifics

- `healthcheck` + `depends_on: condition: service_healthy` — the `api` command (which runs `alembic upgrade head` immediately) will not start until `pg_isready` succeeds, so migrations never race a not-yet-listening Postgres.
- `${VAR:-default}` **syntax** — `ENABLE_SCHEDULER=${ENABLE_SCHEDULER:-true}` etc. Compose substitutes the host env var if set, else the literal default. Without the `:-default`, an unset var would pass an **empty string**, and pydantic parsing `ENABLE_SCHEDULER=""` as `bool` would raise a `ValidationError` at startup.

The defaults make the stack boot with zero host env beyond the three required secrets.

- `--reload + volumes: [".:/app"]` — dev ergonomics: edit a file on the host, uvicorn restarts. This is a **development** default shipped as the only default (section 11).

10.4 Clean-clone requirements

A stranger needs: Docker + Docker Compose, then `cp .env.example .env`, then put **real Stripe test keys** into `STRIPE_SECRET_KEY` / `STRIPE_WEBHOOK_SECRET`, then `docker compose up --build`.

Without Stripe keys: the app still boots (placeholder `sk_test_...` is a non-empty string, so `Settings()` validates). `/generate`, `/usage`, `/health`, the DB, migrations, seeding, quota, cost, and the whole test suite (which mocks Stripe) all work. Only **live** `POST /checkout` (real `stripe.*` calls) and a **real webhook from the Stripe CLI** fail — and `verify_probes.py` sidesteps that by **hand-signing** its webhook with whatever `STRIPE_WEBHOOK_SECRET` is set, so even probes 3–5 pass with a dummy secret as long as it's consistent.

11. Gaps and weak points

Hardcoded values that should be config:

- `app/api/checkout.py` — `success_url` / `cancel_url` are hardcoded `http://localhost:8000/...`. Breaks the moment this isn't on localhost. Should be `settings.BASE_URL` or derived from the request.
- `app/services/stripe_service.py:68` — `unit_amount=4900` (\$49) and `currency="usd"` baked into `_sync_get_or_create_pro_price`. Fine for a demo; a real system would define the price in the Stripe dashboard and only ever `*read*` `STRIPE_PRO_PRICE_ID`.
- `app/services/stripe_service.py` / `webhooks/stripe.py` — `plan_id` defaults to "pro" (`meta_get(..., "plan_id", "pro")`) in several places. If a future third plan existed, a metadata-less event would silently be treated as Pro.
- `app/jobs/scheduler.py:37` — `CronTrigger(hour=3, minute=0)` is not configurable. Minor.
- `docker-compose.yml` — DB credentials `postgres/postgres` and the `5432:5432` host port mapping are literal.
- Port `8000` is assumed by `verify_probes.py`'s default, the checkout URLs, and the compose mapping.

Broad / silent exception handling — is it masking?

- `app/api/checkout.py` except `Exception` -> **acceptable**. It logs the full error server-side and returns a deliberate generic 502. The intent (don't leak Stripe internals, don't 500) is sound. Slightly too broad — a `KeyboardInterrupt/SystemExit` subclass wouldn't be caught (those aren't `Exception`), so it's fine in practice.
- `app/services/stripe_service.py` — no broad catch; Stripe SDK exceptions propagate up to `checkout.py`. Good.
- `app/services/reconciliation_service.py` — except `stripe.error.InvalidRequestError` (narrow, correct: `404 == gone`) and one except `Exception` as `e` around a per-subscription retrieve that **logs a warning and continues**. **Acceptable** — one un-retrievable subscription shouldn't abort the whole sweep — but it *could* swallow a transient network error and leave that tenant un-reconciled until the next night with only a WARNING to show for it. A metric/counter would be better than a log line.
- `app/jobs/runner.py` — except `Exception` in the retry loop is **correct by design**: the whole point is to catch, record, retry, and finally alert. The terminal return `None` (never re-raising) is deliberate so the scheduler survives. The one risk: a bug that raises `BaseException` (not `Exception`) would escape and could kill the scheduler thread silently.
- `app/api/webhooks/stripe.py` — except `IntegrityError` is narrow and correct.

Missing:

- **No CI pipeline**. No `.github/workflows/`, no `Makefile`. `capstone.yaml` names `run/test` commands but nothing runs them automatically on push. The tests-referenced-in-Makefile problem from the prompt template doesn't literally apply (there's no `Makefile`), but the **spirit** does: nothing enforces green on `main`.
- **Migrations are not covered by** `pytest`. `conftest.py` builds the schema with `Base.metadata.create_all`, so a hand-written migration that drifts from the models (they're not `--autogenerated`) would pass every test and only blow up at `alembic upgrade head` in a real deploy. A `pytest` fixture that runs migrations against a scratch DB and diffs against `Base.metadata` would close this.
- **Dev-mode compose is the only mode**. `--reload + bind-mount + postgres/postgres` + a single non-HA Postgres container with a local volume. There's no `docker-compose.prod.yml` or a profile. `README.md` "Limitations" acknowledges the DB side.
- **Single-instance scheduler**. `ENABLE_SCHEDULER=true` on every replica means the nightly job runs once **per replica**. `tech-debt-tracker.md` logs this; the fix is an external scheduler or a leader-election lock.

- **Orphaned** `job_runs`. A process killed mid-job leaves `status="running"` forever; nothing reaps or resumes it.
- `type='api_call'` **is dead weight in the running app**. `UsageEvent.type` supports it, `CostService/UsageQuery` handle it, `tests/test_metering.py` exercises it — but no endpoint ever writes an `api_call` row. It's defensible as "bulk metering hook, tested and ready" but a reviewer will read it as unused code. Either wire a bulk endpoint or delete the branch and simplify `api_calls_used` to `COUNT(*)`.
- `RollupService` **vs** `QuotaService` **on "no subscription"**: **coherent in meaning*, *different in mechanism**. `/generate` -> 402 (hard block). `/usage` -> 200 with `plan_id="none"`, `status="none"` and real usage numbers (disclosure). They agree that a tenant with no subscription is **not** an active Free tenant. They differ because enforcement must refuse and reporting should inform. In normal operation this never triggers — `seed_tenant.py` and every test fixture provision a subscription, and `docs/architecture.md` states the invariant "every Tenant has a subscription row". So: consistent enough to defend, but a nitpicker will note `/usage` returns 200 for a state `/generate` calls a payment error.
- `seed_tenant.py` sets `current_period_end == current_period_start == now()` — an already-expired window (harmless in practice, sloppy).
- `generate.py` **comment numbering** is duplicated (# 3. and # 4. each appear twice) — cosmetic.
- `httpx` is a prod dependency but only used by tests/probes.

As a senior reviewer, the first thing I'd make the author defend or fix:

The `FOR UPDATE` **quota lock's blast radius**. It's the right fix for the boundary race, but it serializes **every** `POST /generate` for a given tenant on that tenant's single subscription row for the **entire** request — including the two `UsageQuery` aggregates and the `MeterService` insert. A high-QPS tenant now has all its `generate` traffic single-file through one row lock, and if the aggregates are slow (they're indexed now, so probably fine, but under lock they're on the critical path) that lock is held longer than it needs to be. Questions I'd ask: (1) Did you measure the throughput ceiling this imposes per tenant? (2) Could the lock be narrowed to just the read-modify-write of the count, or replaced with an atomic `INSERT ... WHERE (SELECT count ...) < limit` / a per-tenant counter row with an atomic `UPDATE ... RETURNING`? (3) What happens to lock wait times when a tenant retries a slow request 5×? The current design trades tenant-level write throughput for correctness — that's the **right** trade for a billing system, but the author should be able to say so explicitly and show they know the cost.

A close second: **no auth at all, including** `/admin/*`. Documented, yes — but "documented" isn't "defensible in production", and `POST /admin/jobs/reconcile` being unauthenticated means any reachable client can spam the Stripe API through your service.

12. One-page mental model (recite this from memory)

What it is: a multi-tenant metering + quota + billing backend. Tenants have one subscription (Free or Pro). Every billable action is metered as an append-only `usage_event`, priced in integer micro-cents, and checked against the plan's monthly allowance **before** it's allowed. Stripe owns billing truth; the local DB is a mirror kept honest by verified webhooks and a nightly reconciliation job.

Lifecycle of one `POST /generate`:

1. FastAPI binds `Idempotency-Key` + `X-Tenant-ID` headers (missing/malformed -> 422) and the `GenerateRequest` body.
2. `generate()` rejects a blank `Idempotency-Key` (400), then checks the tenant exists (404 if not).
3. **Idempotency pre-check:** `SELECT` for `(tenant, key)` in `usage_events`. Hit -> return the **stored** numbers, mirroring the first response. Done. No quota check, no new row.
4. Miss -> parse `mock_usage` (or default to 1 input token); all-zero explicit -> 400.
5. `QuotaService.check_quota(db, tenant, requested_tokens)` — `SELECT ... FOR UPDATE` the subscription row (serializes concurrent callers for this tenant). No sub / not active / unknown plan -> **402**. Compute `api_calls_used` + `tokens_used` via `UsageQuery` over `[period_start, now)`. `current + requested > limit` on either axis -> **429** + `Retry-After`. Nothing has been written.
6. `MeterService.record(...)` — pre-check again; `CostService.price()` computes `cost_microcents` by pricing the 4 token categories separately and summing; `db.add(event)` + `db.flush()`. `IntegrityError` (concurrent same-key) -> rollback, re-`SELECT`, return the winner's row.
7. `await db.commit()` — releases the `FOR UPDATE` lock and persists the event.
8. Respond 200 with `{idempotency_key, tenant_id, text, usage:{api_calls:1, ...token breakdown..., cost_microcents}}`.

Lifecycle of one Stripe upgrade:

1. `POST /checkout` (with `X-Tenant-ID`) -> `StripeService.create_checkout_session` builds a `mode="subscription"` session, `tenant_id` embedded in **both** metadata and `subscription_data.metadata`, using a cached/configured Pro price id. Returns `{session_id, checkout_url}`.
2. Customer pays on Stripe's hosted page with a test card.
3. Stripe POSTs `checkout.session.completed` to `/webhooks/stripe`.
4. Handler: `construct_event` verifies HMAC over the raw body (forged -> **400**, nothing happens). `db.get(WebhookEvent, event.id)` -> seen -> "duplicate", 200. New -> insert the `webhook_events` row.
5. `_handle_checkout_completed`: read `tenant_id` from metadata, `stripe.Subscription.retrieve` the sub for authoritative status + period dates (`get_period_dates`, NULL if absent), then `SubscriptionSync.sync_subscription` upserts the local `subscriptions` row -> `plan_id="pro"`, `status="active"`.
6. One `await db.commit()` writes the `webhook_events` row and the subscription change atomically. `IntegrityError` (concurrent dup) -> "duplicate".
7. Next `GET /usage` -> `RollupService` reads the now-pro subscription -> response shows `plan_id: "pro"` and the higher limits apply on the next `/generate`.

Nightly reconciliation (3 sentences): At 03:00 UTC `APScheduler` calls `run_job("stripe_reconciliation", reconcile_from_stripe, attempts=3)`, which records a `job_runs` row, retries up to 3× with back-off, and on final failure logs CRITICAL "JOB FAILURE ALERT". `reconcile_from_stripe` lists every subscription on the Stripe account, and for each active one repairs any local drift in `stripe_subscription_id` / `plan_id` / `status` / period-end via `SubscriptionSync`. A second pass finds local subscriptions whose Stripe id is no longer active,

re-checks each with `Subscription.retrieve` (a 404 counts as canceled), and downgrades those tenants to Free — catching any `customer.subscription.deleted` webhook that was missed.

The 5 hard parts and the file that owns each:

Hard part	Owned by
Exactly-once metering under retries & concurrency	<code>app/services/meter_service.py</code> (+ <code>uq_tenant_idempotency_key</code>)
Boundary honesty (999 / 1000 / 1001, 402 vs 429, race-safe)	<code>app/services/quota_service.py</code> (+ <code>FOR UPDATE</code> , <code>app/api/errors.py</code>)
Token pricing (cached cheaper, reasoning = output, priced-then-summed, pinned)	<code>app/config/pricing.py</code> + <code>app/services/cost_service.py</code>
Signed + deduplicated Stripe webhooks	<code>app/api/webhooks/stripe.py</code> (+ <code>webhook_events PK</code> , <code>app/services/stripe_helpers.py</code>)
Multi-tenant isolation	every <code>tenant_id</code> -scoped query; <code>app/services/usage_query.py</code> ; proven by <code>tests/test_tenant_isolation.py</code>

Stack in one breath: Python 3.12 · FastAPI + Pydantic v2 · async SQLAlchemy 2.0 + asyncpg · PostgreSQL 16 · Alembic (2 migrations) · Stripe test mode · APScheduler · pytest + pytest-asyncio + httpx · Docker Compose. 6 tables, 7 routes, 7 test modules, 31 tests.